



OXPLiveNG MCP SERVER

Solution Overview & Approval

Prepared for OrthoKinetix

powered by OrthoXpress

Secure AI Access to the OXPLiveNG Claims Database

Prepared by Technijian | 2026-05-26

CONFIDENTIAL — Prepared exclusively for OrthoKinetix / OrthoXpress

01 Executive Summary	3
02 Why Now — The Current-State Gap.....	5
03 The Solution — A Four-Layer Governed Architecture	6
04 What Gets Built.....	8
1 — The Stored-Procedure Layer (in OXPLiveNG)	8
2 — The Swagger / OpenAPI REST API.....	8
3 — The MCP Server.....	8
4 — The MCP Admin Portal.....	9
05 Security, HIPAA & Governance	10
06 Configuration-Driven by Design — Nothing Hardcoded.....	11
07 Delivery Plan & Phases.....	12
08 What We Need From You.....	13
09 Approval & Sign-Off	14
A Appendix A — OXPLiveNG Scope Snapshot	15
Representative Views to Be Exposed.....	15

01 Executive Summary

457

Tables in the OXPLiveNG database

31

Business views to expose safely

1

Governed path: SP to API to MCP

HIPAA

PHI-safe & fully audited by design

OrthoKinetix runs its claims, accounts-receivable, EDI and patient operations on the OXPLiveNG platform — a large, mature SQL Server database with hundreds of tables and dozens of reporting views, used by the OrthoXpress Claims Dashboard. Technijian proposes to make that data safely available to AI assistants (such as Claude) through a Model Context Protocol (MCP) server, so your team can ask plain-language questions — "show me open claims for this payer," "what is the AR balance for this patient" — and get governed, audited answers in seconds.

The work is built on one firm rule that protects the platform: every request travels a single, governed path. AI assistants talk only to a documented API; the API talks to the database only through stored procedures; and a management portal lets your team control exactly what is exposed — with no developer and no code changes. Nothing is hardcoded; every setting lives in the database where it can be changed safely and tracked.

This document explains what we found in OXPLiveNG, what we will build, how it stays HIPAA-safe, the delivery plan, and the approval we are requesting to begin. It is a business-level overview; the full engineering specification is available on request.

What We Are Asking You to Approve

Approval to build the OXPLiveNG MCP Server on the four-layer architecture described in Section 3 — delivered in phases, read-only first, with full create/update/delete added once the read layer is proven.

A short discovery session to confirm which views, data, and operations matter most, and to retire one security gap we found (a database password currently stored in plain text in the application configuration).

Agreement on the phase plan and timeline in Section 7 so Technijian can schedule the work.

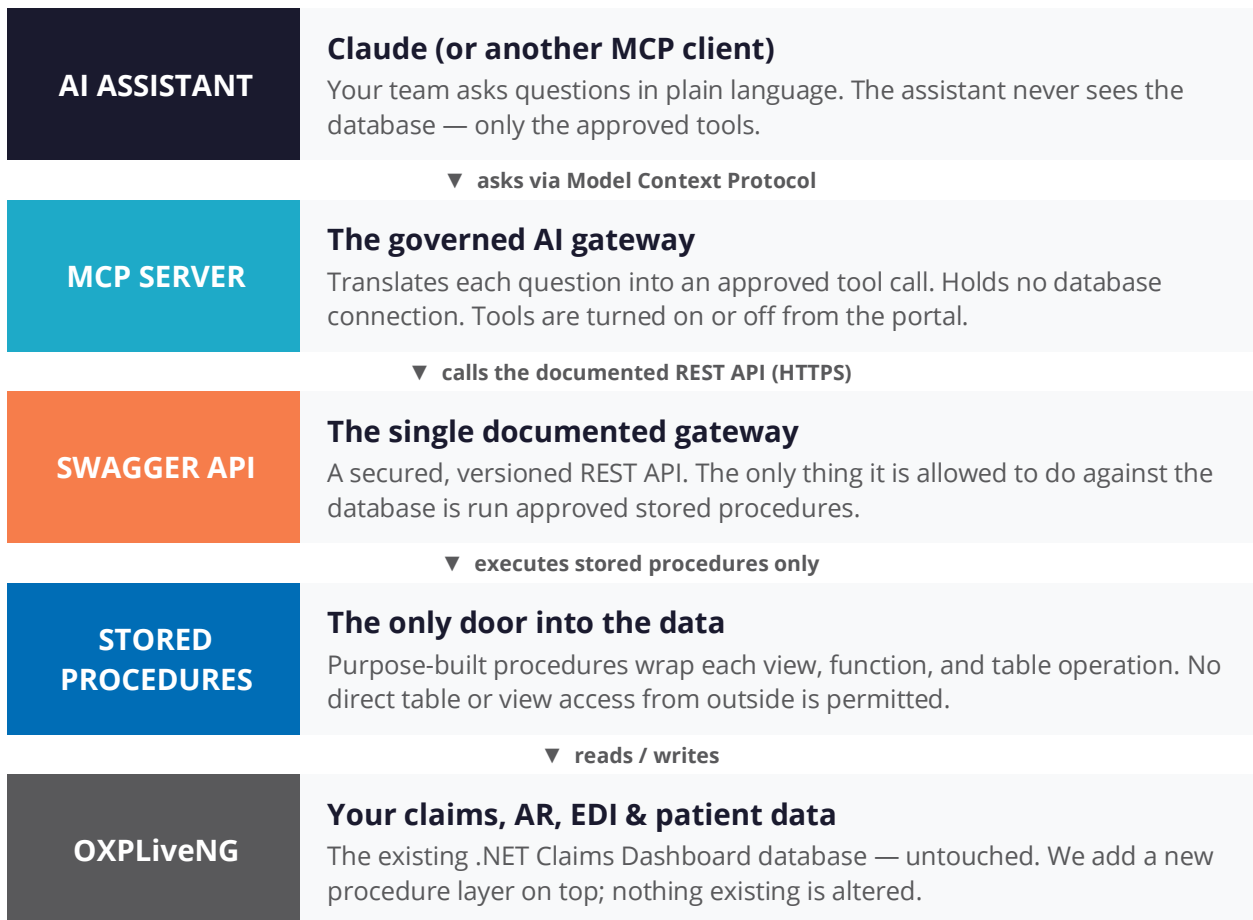
02 Why Now — The Current-State Gap

Technijian reviewed the OXPLiveNG database and the OrthoXpress Claims Dashboard application directly on May 22, 2026. The platform is solid and feature-rich, but the way the application reaches the data today is not ready for safe AI access. The findings below are the reason this project exists — and each one is addressed by the proposed design.

What We Found	Why It Matters	How the MCP Solution Fixes It
No stored-procedure layer exists. Of 6 stored procedures, all 6 are auto-generated SQL Server diagram helpers — zero business procedures.	Applications read tables and views directly, so there is no single, controllable gate for data access — and no safe surface to expose to AI.	We build a governed stored-procedure layer that becomes the only way in (Section 4).
The Claims Dashboard reads data directly via its data layer (Entity Framework 6, database-first), not through procedures.	Logic and access rules are scattered across the app instead of centralized and auditable.	The API and AI both call procedures, centralizing rules, validation, and auditing.
A database password is hardcoded in plain text in the application configuration (web.config).	This is a security and compliance exposure for a system holding patient data (PHI).	Replaced with a least-privilege login whose secret is stored in a secure vault — never in code (Section 5).
31 business views hold valuable, ready-to-use information (claims dashboard, transactions, AR balances, payer risk, payment-notification metrics).	This is exactly the data AI should surface — but only through a safe, read-controlled channel.	Each view is wrapped in a procedure, exposed as an API endpoint, and offered as an AI tool.
The physical database is cluttered with backup and temporary tables (dated backups, Temp_copies).	Direct access risks exposing the wrong or stale data to users and AI.	The API surface is a curated allow-list; clutter is excluded by design.

03 The Solution — A Four-Layer Governed Architecture

Every AI request follows the same one-way path, top to bottom. Each layer has one job and can only talk to the next — so there is exactly one controllable, auditable route to your data. This is the same proven architecture Technijian already runs in production for other ORX systems.



The Admin Portal — You Stay in Control

A separate management portal lets your team operate the MCP server with no developer: enable or disable tools, manage who can access what, review a full audit trail, schedule jobs, set data-retention rules, and test any tool safely.

Because every setting lives in the database, changes are immediate, tracked, and reversible — there is no code to edit and no redeployment to wait for.

04 What Gets Built

Four components make up the delivery. Each is a discrete, testable piece, and each maps directly to a layer in Section 3.

1 — The Stored-Procedure Layer (in OXPLiveNG)

- Read procedures that wrap all 31 business views and the key reporting functions (e.g., the claims-dashboard filter and the patient-balance lookup).
- Full create / read / update / delete procedures for the core entities — claims, claim services, patients, insurance, jobs/orders and claim status notes — generated from the database so the set grows by configuration, not custom code.
- A self-describing catalog: small tables that list every exposed procedure, its inputs, and its outputs — so the API and AI tools are generated from the database, never hand-maintained.

2 — The Swagger / OpenAPI REST API

- A secured .NET 8 API that exposes one endpoint per approved procedure, fully documented in Swagger.
- Sign-in with Microsoft Entra ID (Azure AD), role-based permissions, input validation, paging, and standardized error messages.
- Patient-data fields are flagged and hidden from anyone without explicit clearance.

3 — The MCP Server

- Connects to AI assistants two ways: directly in Claude Desktop/Code, and over the web for remote use.
- Exposes each approved API endpoint as an AI tool, plus ready-made prompts for common questions (find claims by patient, AR summary by payer, add a claim status note).

- Talks only to the API — it has no path to the database — and every tool call is logged.

4 — The MCP Admin Portal

- A web dashboard for your team to manage tools, connections, users and roles, the audit log, schedules, notifications, data retention, and a built-in test runner.
- All controlled through the database, so the people who run the platform do not need a developer to make changes.

05 Security, HIPAA & Governance

OXPLiveNG contains protected health information — patients, insurance, and EDI 835/837 claims data. Security is therefore designed into every layer, not added afterward. The controls below are standard in this architecture.

Control	What It Means for OrthoKinetix
One governed path	AI and applications can reach data only through approved stored procedures behind the API — there is exactly one route to audit and control.
Least privilege	The API signs in with an account that can only run approved procedures — it cannot read or change tables directly.
No hardcoded secrets	Passwords and keys live in a secure vault and are referenced, never stored in code or config. The current plain-text password is retired.
Minimum necessary (PHI)	Patient-data fields are tagged; users and AI tools without clearance never receive them.
Full audit trail	Every tool call, API call, and procedure run is logged with who, what, when, and a correlation id — with patient data redacted in the logs.
Deny by default	Nothing is exposed unless explicitly enabled and permitted for that role. Write operations are off until you turn them on.
Encrypted everywhere	All connections use TLS; database connections are encrypted.

This design supports your HIPAA compliance posture; it does not replace your organization's policies, business-associate agreements, or staff training, which remain in place.

06 Configuration-Driven by Design — Nothing Hardcoded

A specific requirement of this project is that the system is driven entirely from the database — not from code or configuration files. This is what keeps OrthoKinetix in control and the system easy to operate over time.

Stored in the database (changeable any time)	The benefit
Which procedures, endpoints, and AI tools exist and are enabled	Add or retire a capability by changing data — no code release.
API addresses, connection settings, and timeouts	Re-point or tune the system without touching code.
Users, roles, and permissions	Manage access centrally with a full change history.
Schedules, notification rules, and data-retention policy	Operations teams adjust behavior themselves.
Secret references (not the secrets themselves)	Sensitive values stay in a vault; the database only points to them.

Why This Matters

When everything that changes lives in data, your team can adapt the system safely and instantly — and every change is tracked. There are no hidden settings buried in code, and no waiting on a developer to flip a switch.

07 Delivery Plan & Phases

The work is delivered in six phases. Read-only access is proven first, which lets your team see value early and with the lowest possible risk; full create/update/delete is added only once the read layer is solid. Each phase ends with a clear, demonstrable result.

Phase	Deliverable	You Will See
0	Foundations — project setup, secure login, configuration tables, retire the hardcoded password	A clean, secure footing; the plain-text password gone
1	Read procedure layer over all 31 views and key functions	Every report view reachable through the governed layer
2	Swagger API for all read operations	A live, documented API your team can explore
3	MCP server (read) + Admin Portal first release	Claude answering real questions; the portal toggling tools
4	Full create / update / delete for core entities	AI and apps can safely update data through procedures
5	Hardening — retention, alerts, testing, HIPAA review	Production-ready, reviewed, and signed off

Detailed timeline and effort estimates are confirmed during discovery (Section 8) and provided in a companion Statement of Work upon approval.

08 What We Need From You

To begin, Technijian needs a short discovery session and a few decisions. None of these block approval — they shape the detailed plan.

- ☐ Priority data: which views, reports, and operations matter most to surface first.
- ☐ Write scope: confirm the core entities that should support create/update/delete in Phase 4.
- ☐ Access & roles: who should use the AI tools, and at what permission level (including who may see patient data).
- ☐ Environment: confirmation to use the existing test database (OXPTestDB / UAT) for build-and-test before production.
- ☐ Security: approval to retire the hardcoded credential and move secrets into a vault.
- ☐ Hosting & SSO: confirm Microsoft Entra ID (Azure AD) for sign-in and the preferred hosting (on-premises IIS or container).

09 Approval & Sign-Off

By signing below, OrthoKinetix authorizes Technijian to proceed with the OXPLiveNG MCP Server as described in this document, beginning with Phase 0 and the discovery session. This approval covers the approach and phase plan; specific scope, timeline, and fees are confirmed in a companion Statement of Work.

Role	Name & Signature	Date
OrthoKinetix — Authorized Approver	_____ Name & Signature	_____ Date
OrthoKinetix — Technical / Security	_____ Name & Signature	_____ Date
Technijian — Engagement Lead	_____ Name & Signature	_____ Date

Next Step

On approval, Technijian schedules the discovery session, issues the Statement of Work with timeline and fees, and begins Phase 0. Questions in the meantime: your Technijian engagement lead, or 949.379.8500.

A Appendix A — OXPLiveNG Scope Snapshot

Captured live from the OXPLiveNG database on May 22, 2026. Provided for transparency; final scope is confirmed during discovery.

Object Type	Count	Role in This Project
Business views	31	Primary read targets — wrapped in procedures and exposed as AI tools
Functions	12	Key reporting logic (claims-dashboard filter, patient balance) used inside procedures
Tables	~457	The data estate; core entities receive full CRUD, clutter is excluded
Business stored procedures	0	None exist today — the governed layer is built new (the heart of this project)

Representative Views to Be Exposed

vw_ClaimDashboard, vw_ClaimTransaction, vw_ClaimService / vw_ClaimServiceBalance / vw_ClaimServiceAggregated, vw_ARSummaryPRBalance, vw_ClaimHistory, vw_GetClaims, vw_RiskPayerGroup, vw_DuplicateClaimsView, and the vw_Notification* payment-metric set — among the full set of 31 views in OXPLiveNG.

About Technijian

Technijian is an Irvine, California IT and AI services firm delivering managed IT, cybersecurity, custom application development, and AI/agent integration. This MCP architecture is already in production for other ORX systems — OrthoKinetix benefits from a proven pattern, not an experiment.